

PYTHON 1

- Rappels des commandes usuelles -

I. Rappels de quelques commandes incontournables

La commande Python	a pour effet de :
<code>x=input()</code> ou <code>x=input('valeur ?')</code> ou <code>x=input('valeur de x = ?')</code> ...	créer une variable Python nommée <code>x</code> , dont la valeur est celle tapée au clavier
<code>print(x)</code>	afficher dans la console la valeur contenue dans la variable Python nommée <code>x</code>
<code>print('x')</code>	afficher dans la console : <code>x</code>
<code>print('x=',x)</code>	afficher dans la console <code>x=</code> , suivi de la valeur de la variable Python nommée <code>x</code>
<code># commentaires</code>	ajouter des commentaires : tout ce qui est écrit à la suite de <code>#</code> n'étant pas exécuté en tant qu'instructions.
<code>range(n)</code> , où <code>n</code> est une variable Python à valeurs dans $\mathbb{N}^*$. Ou aussi : <code>range(5)</code> , <code>range(100)</code> ,...	créer une liste (de type <code>range</code>) dont les éléments sont $0, 1, \dots, n-1$. La commande <code>range</code> sera particulièrement utile pour les boucles itératives <code>for</code>
<code>range(i,j)</code> , où <code>i</code> et <code>j</code> sont deux variables Python à valeurs dans $\mathbb{Z}$ de valeurs telles que $i < j$	créer une liste (de type <code>range</code>) dont les éléments sont $i, i+1, \dots, j-1$.
<code>range(i,j,k)</code> , où <code>i</code> , <code>j</code> , <code>k</code> sont trois variables Python à valeurs dans $\mathbb{Z}$ de valeurs telles que $k > 0$ et $i < j$	créer une liste (de type <code>range</code>) dont les éléments sont $i, i+k, i+2k, \dots, i+pk$ où p est le plus grand entier positif tel que $i+pk < j$.

II. Les premiers types de variables en Python

Les premières variables en Python manipulées sont du type suivant :

- `int` (pour « integer »), c'est-à-dire une variable à valeurs dans $\mathbb{Z}$,
- `float`, c'est-à-dire une variable à valeurs dans $\mathbb{R}$,
- `bool` (pour « boolean »), c'est-à-dire une variable de valeur `True` ou `False`,
- `str` (pour « string »), c'est-à-dire une variable définie comme une chaîne de caractère (encadrée par des « ' » ou « " »).

Par exemple, les commandes suivantes en Python :

```
n=2
x=2.
y=2/3
S= 'Bonjour'
t=True

print(n)
print(y)
print(type(n))
print(type(x))
print(type(S))
print(type(t))
```

donnent dans la console après compilation :

```
runcell(1, 'C:/Users/ccarc/Documents/intro_python.py')
2
0.6666666666666666
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Le symbole Python =

Le symbole Python = n'a pas la même signification qu'en mathématiques.

Par exemple, la commande Python précédente `n=2` est une affectation, et se lit de gauche à droite : ici, on affecte à la variable Python notée `n`, la valeur de 2.

Dans ce cas, Python crée une case mémoire appelée `n`, et la valeur de cette case mémoire est 2.

Cette action revient à la phrase mathématique : « Soit $n = 2$. ».

II.1. Opérations sur les réels (ou les entiers)

- La somme, différence, produit et division sont respectivement notées en Python : +, -, * et /.
- La fonction puissance est notée ** en Python. Par exemple, la commande `x=2**3` affecte à la variable Python `x` la valeur 8.
- La commande `abs(x)` renvoie la valeur absolue de la variable `x`.
- Pour utiliser les fonctions mathématiques usuelles, on écrira en préambule :

```
import math as m
```

Les fonctions mathématiques usuelles seront précédées de `m.` pour leur utilisation en Python, comme ci-dessous :

La commande Python	affecte à la variable Python <code>x</code> la valeur :
<code>x=m.sqrt(t)</code>	$\sqrt{t}$
<code>x=m.exp(t)</code>	e^t
<code>x=m.log(t)</code>	$\ln(t)$
<code>x=m.sin(t)</code>	$\sin(t)$
<code>x=m.cos(t)</code>	$\cos(t)$
<code>x=m.tan(t)</code>	$\tan(t)$
<code>x=m.floor(t)</code>	$\lfloor t \rfloor$ (la partie entière de t)
<code>x=m.factorial(n)</code>	$n!$

De même, le module Python `math` permet d'accéder aux constantes e et π , notées respectivement en Python : `m.e` et `m.pi`.

Remarque :

La commande Python `int` permet, lorsque cela est possible, de transformer un réel en un entier.

Par exemple, les commandes ci-dessous :

```
x=m.sqrt(16)
print(type(x))
m=int(x)
print(type(m))
```

```
u = '3'
print(type(u))
v = int(u)
print(type(v))
```

donnent l'écran de sortie suivant :

```
runcell(2, 'C:/Users/ccarc/Documents/intro_python.py')
<class 'float'>
<class 'int'>
<class 'str'>
<class 'int'>
```

De même, la valeur d'une variable Python de type `int` ou `float` peut être transformée en chaîne de caractère à l'aide de la commande `repr`.

Les opérations mathématiques ci-dessus peuvent également s'appliquer à des listes ou à des matrices à coefficients réels : elles sont alors effectuées coefficient par coefficient.

II.2. Opérations sur les booléens

- Les opérateurs logiques mathématiques ET, OU, NON sont respectivement notés en Python :

`and`, `or`, `not`.

On a ainsi, lorsque `b1` et `b2` sont deux variables Python de type `bool` :

La commande Python	affecte à la variable Python <code>c</code> la valeur :
<code>c = b1 and b2</code>	$\begin{cases} \text{True} & \text{si } b1 \text{ et } b2 \text{ contiennent la valeur True} \\ \text{False} & \text{sinon} \end{cases}$
<code>c = b1 or b2</code>	$\begin{cases} \text{False} & \text{si } b1 \text{ et } b2 \text{ contiennent la valeur False} \\ \text{True} & \text{sinon} \end{cases}$
<code>c = not(b1)</code>	$\begin{cases} \text{False} & \text{si } b1 \text{ contient la valeur True} \\ \text{True} & \text{sinon} \end{cases}$

- Les symboles Python

`==` (test d'égalité), `!=` (test de non égalité), `<`, `<=`, `>`, `>=`

sont des tests (effectués entre deux réels, ou deux listes ou matrices de même taille). Le résultat d'un test en Python est soit le booléen `True` si l'assertation mathématique associée est vraie, et `False` sinon.

Les tests seront particulièrement utiles pour les structures conditionnelles `if ....` : et `elif ....`, ou pour les boucles itératives `while.....`.

Voici des exemples d'utilisation simples des booléens, écrits dans la console :

```
1==1
Out[170]: True

x=1

x==1
Out[172]: True

x!=2
Out[173]: True

y=3

x>y
Out[175]: False

(x>0) and (x<y)
Out[176]: True
```

Les opérations mathématiques ci-dessus peuvent également s'appliquer à des listes ou à des matrices à coefficients réels : elles sont alors effectuées coefficient par coefficient.

III. Le type list

Une variable `L` de type `list` en Python est une liste ordonnée d'un nombre fini d'éléments donnés entre crochets, les différents éléments de la liste étant séparés par une virgule.

Rangs (ou index) des éléments d'une liste

Python numérote les éléments d'une liste :

- de gauche à droite en commençant par 0, et en ajoutant +1 au rang de chaque nouvel élément,
- de droite à gauche en commençant par -1, et en ajoutant -1 au rang de chaque nouvel élément.

Ainsi, si `L` est une liste en Python, à n composantes, alors :

- ses éléments sont numérotés de gauche à droite de 0 à $n - 1$,
- ses éléments sont numérotés de droite à gauche de -1 à $-n$.

Par exemple, les commandes ci-dessous définissent des variables de type `list` en Python :

```
L=[1,2,3]
L1=[1,2,"a"]
L2=[k for k in range(10)]
# ou encore :
L2bis=list(range(10))
L3=[1,2]*4
# pour une liste M,
# l'opération M*n recopie n fois à la suite la liste M

print('L=',L)
print('L1=',L1)
print('L2=',L2)
print('L3=',L3)
```

Après compilation des commandes, la console affiche :

```
runcell(2, 'C:/Users/ccarc/Documents/intro_python.py')
L= [1, 2, 3]
L1= [1, 2, 'a']
L2= [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
L3= [1, 2, 1, 2, 1, 2, 1, 2]
```

Quelques opérations sur les listes

Dans le tableau ci-dessous, on suppose que :

- L , $L1$ et $L2$ sont trois variables Python de type `list`,
- x une variable Python de type quelconque,
- i , j et k sont trois entiers adaptés aux index des listes.

La commande Python	a pour effet de :
<code>len(L)</code>	renvoyer le nombre de composantes de L
<code>u=L[i]</code>	affecter à la variable Python u la valeur de la composante de L située au rang i
<code>L[i]=x</code>	changer la liste L en remplaçant l'élément situé au rang i par la valeur contenue dans la variable Python x .
<code>M=L[i:j]</code>	affecter à la variable Python M la sous-liste de L dont les éléments sont les composantes de L situées du rang i au rang $j-1$ inclu
<code>L.append(x)</code>	ajouter l'élément x à droite dans la liste L
<code>L.insert(k,x)</code>	ajouter l'élément x dans la liste L au rang k
<code>u=L.pop(i)</code>	enlever l'élément de la liste L situé au rang i et d'affecter sa valeur dans la variable u
<code>u=L.pop()</code>	enlever le dernier élément de la liste L (celui à droite) et d'affecter sa valeur dans la variable u
<code>L.count(x)</code>	renvoyer le nombre de composantes de la liste L égales à x
<code>L.index(x)</code>	renvoyer le plus petit rang positif où est situé l'élément x dans la liste L si x est bien un élément de L
<code>M=L1+L2</code>	affecter à la variable Python M la liste obtenue en « concaténant » les listes $L1$ et $L2$: les premiers éléments de M sont ceux de la liste $L1$, et les éléments suivants de M sont ceux de $L2$.

IV. Le type array

Pour construire des vecteurs ou des matrices en Python, on commence par charger le module `numpy`, via la commande :

```
import numpy as np
```

Numérotation Python des éléments d'une matrice

Comme pour les listes, les éléments d'une matrice (respectivement d'un vecteur) définie en Python (variable de type `array`), sont numérotés pour les lignes :

- de gauche à droite en commençant par 0, et en ajoutant +1 au rang de chaque nouvel élément,
 - de droite à gauche en commençant par -1, et en ajoutant -1 au rang de chaque nouvel élément ;
- et sont numérotés pour les colonnes :
- de haut en bas en commençant par 0, et en ajoutant +1 au rang de chaque nouvel élément,
 - de bas en haut en commençant par -1, et en ajoutant -1 au rang de chaque nouvel élément.

Créations de matrices en Python

Pour créer en Python :

- le vecteur $(x_1, \dots, x_n)$ de $\mathbb{R}^n$, on utilise la commande `x=np.array(X)` où `X` est la liste (variable de type `list`) de composantes $x_1, x_2, \dots, x_n$,
- la matrice $M = (m_{i,j})_{\substack{1 \leq i \leq p \\ 1 \leq j \leq q}}$ de $\mathcal{M}_{p,q}(\mathbb{R})$, on utilise la commande `M=np.array([L1,L2,...,Lp])` où pour tout $i \in \llbracket 1, p \rrbracket$, `Li` est la liste (variable de type `list`) dont les composantes sont $m_{i,1}, m_{i,2}, \dots, m_{i,q}$.

→ Lorsque l'entier n est petit, on entre à la main les composantes du vecteur ou de la matrice. Par exemple, les commandes

```
x=np.array([0,1,0,1])
M=np.array([[1,2,3],[3,1,2],[2,3,1]])
```

définissent respectivement le vecteur $x = (0, 1, 0, 1)$ et la matrice $M = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \\ 2 & 3 & 1 \end{pmatrix}$.

→ Lorsque n représente un entier naturel strictement positif, contenu dans une variable Python `n`, pour définir le vecteur $x = (x_1, \dots, x_n)$ en Python, on commencera par initialiser une variable Python `x` sous la forme :

	qui affecte à la variable <code>x</code> :
<code>x=np.zeros(n)</code>	le vecteur nul de $\mathbb{R}^n$
<code>x=np.ones(n)</code>	le vecteur de $\mathbb{R}^n$ dont toutes les composantes sont égales à 1
<code>x=np.arange(n)</code>	le vecteur $(0, 1, 2, \dots, n-1)$ de $\mathbb{R}^n$
<code>x=np.arange(i,i+n)</code> , avec <code>i</code> un réel ou un entier	le vecteur $(i, i+1, i+2, \dots, i+n-1)$ de $\mathbb{R}^n$

puis on changera la valeur des composantes de la variable `x`, selon les besoins :

- à la main pour les valeurs particulières, avec une commande de la forme `x[i]=***`, la variable Python `x[i]` contenant la valeur de la composante du vecteur `x` située au rang `i`,
- à l'aide d'une boucle `for`, pour les valeurs génériques.

→ Lorsque n représente un entier naturel strictement positif, contenu dans une variable Python n , pour définir la matrice carrée $M = (m_{i,j})_{1 \leq i,j \leq n}$ en Python, on commencera par initialiser une variable Python M sous la forme :

	qui affecte à la variable M :
<code>M=np.zeros([n,n])</code>	la matrice nulle de $\mathcal{M}_n(\mathbb{R})$
<code>M=np.ones([n,n])</code>	la matrice de $\mathcal{M}_n(\mathbb{R})$ dont toutes les composantes sont égales à 1
<code>M=np.eye(n)</code>	la matrice unité I_n de $\mathcal{M}_n(\mathbb{R})$

puis, comme pour les vecteurs, on changera la valeur des composantes de la variable M , selon les besoins.

→ Lorsque p et q représentent deux entiers naturels strictement positifs, contenus dans deux variables Python p et q , pour définir la matrice carrée $M = (m_{i,j})_{\substack{1 \leq i \leq p \\ 1 \leq j \leq q}}$ en Python, on commencera par initialiser une variable Python M sous la forme :

	qui affecte à la variable M :
<code>M=np.zeros([p,q])</code>	la matrice nulle de $\mathcal{M}_{p,q}(\mathbb{R})$
<code>M=np.ones([p,q])</code>	la matrice de $\mathcal{M}_{p,q}(\mathbb{R})$ dont toutes les composantes sont égales à 1

puis, comme pour les vecteurs, on changera la valeur des composantes de la variable M , selon les besoins.

Extraction d'éléments d'un vecteur ou d'une matrice

On suppose prédéfinis :

- x une variables Python de type **array**, représentant un vecteur $(x_0, x_1, \dots, x_{n-1})$ de $\mathbb{R}^n$,
- M une variable Python de type **array**, représentant une matrice carrée ou rectangulaire $M = (m_{i,j})_{\substack{0 \leq i \leq p-1 \\ 0 \leq j \leq q-1}}$ à coefficients réels,
- i, j, k des variables Python de type **int**, dont les valeurs sont adaptées à la taille de x ou à celle de M .

La commande Python	affecte à la variable u :
<code>u=x[i]</code>	le coefficient situé au rang i du vecteur x
<code>u=x[i:j]</code>	le vecteur de coefficients $x_i, x_{i+1}, \dots, x_{j-1}$
<code>u=M[i,j]</code>	le coefficient $m_{i,j}$ situé à la ligne i et colonne j de la matrice M
<code>u=M[:,j]</code>	le vecteur de $\mathbb{R}^p$ dont les coefficients sont ceux de la $j^{\text{ième}}$ colonne de M
<code>u=M[i,:]</code>	le vecteur de $\mathbb{R}^q$ dont les coefficients sont ceux de la $i^{\text{ième}}$ ligne de M
<code>u=M[i:j,:]</code>	la sous-matrice constituée des lignes n° i à $j-1$ de M
etc...	

Quelques opérations sur les vecteurs et matrices

Les opérations mathématiques usuelles et les tests peuvent s'appliquer à des matrices, et sont dans ce cas effectués coefficient par coefficient.

On suppose prédéfinis :

- $\mathbf{x}$ une variable Python de type `array`, représentant un vecteur $(x_0, x_1, \dots, x_{n-1})$ de $\mathbb{R}^n$,
- A et B deux variables Python de type `array`, représentant deux matrices carrées ou rectangulaires à coefficients réels,

La commande Python	renvoie :
<code>len(x)</code>	le nombre de composantes du vecteur $\mathbf{x}$
<code>len(A)</code>	le nombre de lignes de A .
<code>p,q=np.shape(A)</code>	le nombre de lignes (valeur de la variable p) et de colonnes (valeur de la variable q) de A .
<code>np.dot(A,B)</code>	le produit matriciel des deux matrices A et B , en supposant que le nombre de lignes de B est égal au nombre de colonnes de A .
<code>np.transpose(A)</code>	$A^\top$, la transposée de la matrice
<code>min(x)</code> (resp. <code>np.min(A)</code>)	la plus petite valeur parmi les coefficients de $\mathbf{x}$ (resp. de A)
<code>max(x)</code> (resp. <code>np.max(A)</code>)	la plus grande valeur parmi les coefficients de $\mathbf{x}$ (resp. de A)
<code>sum(x)</code> (resp. <code>np.sum(A)</code>)	la somme de tous les coefficients de $\mathbf{x}$ (resp. de A)

V. Programmation

V.1. Définition d'une fonction en Python

Une fonction en Python se définit comme suit (parfois la variable Python y pourra être omise) :

```
def nom de la fonction(liste éventuelle des paramètres de la fonction):  
    # liste des instructions à effectuer  
    # permettant de calculer la valeur de  $y$   
    return(y)
```

Une fonction Python s'utilise comme en mathématiques, une fois qu'elle a été définie.

V.2. Structure conditionnelle : if

Dans les exemples ci-dessous, on utilise la commande `rd.randint(a,b)`, en ayant préalablement tapé la commande `import numpy.random as rd`, qui renvoie un entier parmi $a, a+1, \dots, b-1$ avec équiprobabilité, lorsque a et b représentent deux entiers a et b tels que $a < b$.

- Un premier exemple (commande `else` omise).

Le script ci-contre renvoie la simulation d'une variable aléatoire X de loi uniforme sur $\{-1, 1\}$.

```
X=1  
if rd.randint(0,2)==0:  
    X=-1
```

- Un second exemple (structure « `if... else` »).

Le script ci-contre renvoie également la simulation d'une variable aléatoire X de loi uniforme sur $\{-1, 1\}$.

```
if rd.randint(0,2)==0:  
    X=-1  
else:  
    X=1
```

- Un second exemple (structures « `if` » imbriquées).

Le script ci-contre renvoie la couleur de la boule piochée, lorsque l'on tire une boule dans une urne contenant une boule jaune, deux boules vertes et trois boules bleues.

```
X=rd.randint(1,7)  
if X==1:  
    c='jaune'  
elif X<=3:  
    c='vert'  
else:  
    c='bleu'  
print(c)
```

V.3. Structure répétitive à nombre de boucles connu : `for`

Une boucle `for` se construit comme suit :

```
for k in X:
    # Liste des commandes à effectuer
    # pour chaque valeur de k variant dans l'ensemble X
```

l'ensemble `X` pourra être de la forme `range(n)`, `np.arange(1,n+1)`, un vecteur `X` de $\mathbb{R}^n$, une liste...

Par exemple, le script ci-contre renvoie la valeur

de $\sum_{k=1}^n k e^{-k}$, pour $n = 10$.

```
n=10
S=0
for k in range(1,n+1):
    S=S+k*m.exp(-k)
```

V.4. Structure répétitive à nombre de boucles inconnu : `while`

Une boucle `while` se construit comme suit :

```
while test à effectuer :
    # Liste des commandes à effectuer
    # tant que le résultat du test est True
```

Par exemple, le script ci-contre renvoie la valeur du plus petit entier naturel n tel que

$$\left| \sum_{k=0}^n \frac{1}{k!} - e \right| \leq 10^{-3}.$$

```
n=0
S=1
p=1
while abs(S-m.e)>0.001:
    n=n+1
    p=p*n
    S=S+1/p
print(n)
```